

Agentic Commerce Platform

A marketplace where the buyer is an agent — and every rupee it moves needs a human yes.

503 products · 21 merchants

4,008 reviews · 33 tables

224 tests passing

REFERENCE DOCUMENT

A marketplace built for buyers that aren't people

Merchants publish once and become discoverable and transactable by AI buyers. A shopper describes what they want in plain language; an agent searches every merchant's machine-readable catalogue, ranks the options against published criteria, explains its choice from the numbers that produced it, and stops dead before it spends anything.

503

PRODUCTS

21

MERCHANTS

4,008

REVIEWS

5

AGENT PROTOCOLS

₹0

SPENT WITHOUT CONSENT

The one idea everything else follows from

The language model is not the search engine, the ranker, or the executor. It is the interpreter and the narrator.

It reads what a shopper meant and writes back what the system decided. Between those two jobs sit ordinary, testable functions: Postgres does retrieval, a weighted scorer does ranking, a policy engine decides whether money may move. The model never chooses the product, never sets the order, and never authorises a payment.

That split isn't caution for its own sake. Because the explanation is generated *from* the score vector, it cannot invent a reason the ranker didn't use — and because ranking is a pure function, the same catalogue and the same query give the same answer every time.

A worked consequence. Early on the model guessed the category “Activewear” for a yoga mat. Categories are a hard filter, so that guess removed every yoga mat and surfaced t-shirts instead. The fix wasn't a better prompt — it was making category rule-owned. Any field where a wrong value changes what a shopper is *allowed to see or buy* is now decided in code.

How one shopping turn works

Two model calls per turn, not twenty. The order is fixed, each stage has one job, and every stage writes to an append-only audit log — so a turn can be replayed rather than guessed at.

1	MODEL	Understand Reads the whole conversation. Returns what it understood, what is still unknown, and the next question in its own words. Anything unstated comes back <code>null</code> — never guessed.
2	CODE	Retrieve pgvector cosine + Postgres full-text, fused by Reciprocal Rank Fusion (k=60), then hard filters for size, colour, budget, stock, sale window and rated features.
3	CODE	Rank Seven weighted criteria. Every one reports <code>weight × normalised = contribution</code> , so the ranking shows its working.
4	MODEL	Explain Turns that score vector into three to five plain sentences. It is given the criteria and forbidden from adding a fact that isn't in them.
5	CODE	Authorise Policy engine returns a verdict with the exact bound that produced it. Signed mandate chain verified immediately before money is requested.
6	CODE	Pay & confirm Gateway signature verified, mandate chain re-verified, stock committed, order settled. A failed check charges nothing.

Shopping

It asks rather than assumes

“I want shoes” matches most of the catalogue, so ranking it would be arbitrary advice. The agent asks — purpose, then size, then colour, then budget — and only then ranks. Rules decide *which* slot is missing and what the options are; the model only phrases the question. Options come from live stock, so tapping “Black” always leads to black shoes you can buy today, and a skipped answer stays null rather than becoming a silent filter.

There is no keyword matching on your replies. “*Something for pounding the pavement on weekends, building up to a half marathon*” becomes a search for road running shoes because the model understood it.

You can see and change what it optimised for

A ranking whose priorities are hidden can't be argued with, so the weights are shown as an order you can rearrange — click the ▲ ▼ arrows on any row, or drag it, then re-rank. The default now reads:

PRIORITY	WEIGHT	WHY IT SITS THERE
Matching your request	22%	Not a preference — it's what keeps results about the thing you asked for, so it isn't reorderable
Rating	22%	Bayesian-shrunk, so 5.0 from three reviews can't beat 4.6 from nine hundred
Seller reliability	19%	A great product from a seller who doesn't ship is worth less than a good one that arrives
Price	18%	Matters, but no longer outranks whether the order will be honoured
Delivery speed	7%	Merchant's standard dispatch window
Stock	6%	Less chance of a cancellation
Return window	6%	Longer and easier to send back

When the exact thing doesn't exist, it sells the nearest thing

Two empty results look identical and deserve opposite treatment. If the marketplace doesn't stock that *kind* of thing — a gaming laptop — it says so and offers nothing, because offering shoes instead is exactly the hallucination the relevance gate exists to prevent. If it stocks the kind but a filter removed every option — size 15, purple formal shoes — it offers the closest buyable alternatives, each labelled with how it differs: *"no size 15 — available in 7, 8, 9, 10, 11."* Those availability claims are queried from live variants, never inferred.

The rest of the shopping surface

Rated features

Every product carries 1–5 scores for the qualities its category is measured on — breathability, grip, water resistance, battery life — shown as bars with the number beside them.

One checkout, many merchants

Baskets stay per-merchant because fulfilment and signed cart mandates are. You approve once and pay once; each merchant still gets its own order.

Cross-sell that isn't more of the same

Real co-purchase first, then complementary categories. Formal shoes suggest trousers, socks and shirts — not another pair of formal shoes.

Reviews you can read

Verified-purchase only, enforced in the schema. Product pages show the rating histogram, because "4.2 from 900" hides whether that's agreement or a split.

Agent activity

An append-only timeline of every step, with the scoring breakdown and every refusal, including the ones you never saw.

Spending limits

Per-order and per-day ceilings, plus a saved test payment method that stores no credentials at all.

Selling

A merchant publishes once. There is no separate “publish to AI” step to forget — every write re-indexes the product, so the AI-readable catalogue, the ACP feed and the embeddings can’t drift from what was just saved.

Dashboard	Revenue over 30 days / 12 months / 5 years, best sellers, category mix, sparkline stat tiles, and inventory alerts ranked by urgency rather than alphabetically.
Listing wizard	Item → brand → product, with specifications and search tags fetched automatically. Manual fallback at every step, and provenance labelled so a merchant knows what was suggested versus what they typed.
Search tags	Their own column at full-text weight A against description weight B. Measured: the same term ranks 0.6079 as a tag against 0.2432 in body copy — 2.5×. Editable and regenerable.
Insights agent	Six detectors — stockout, stockout risk, overstock, demand drop, catalogue quality, stale-unavailable — each with evidence, projected impact and a confidence. Policy-gated and queued for approval; it proposes, it never acts.
Orders	Mark delivered, cancel with stock returned, or refund through the rails the charge actually arrived on.
Reviews	Everything customers wrote, with a “needs attention” filter for three stars and below — a feed that opens on the praise is pleasant and useless.
Store settings	Profile, return and shipping policies, and the agent limits that bound what an automated buyer may do with this store.

Money, and the consent in front of it

Every money-moving or catalogue-mutating action passes through one policy engine before it happens. It returns a verdict, a human-readable reason, and the exact bound that produced it — so a refusal is as auditable as a completion.



Platform limits are a *ceiling*, not a default: a per-user or per-merchant rule can only ever be stricter, otherwise raising your own limit would let you escape the platform bound. The shipped default sets `requireApprovalAboveMinor: 0` — every payment needs an explicit human yes.

The mandate chain

Three signed documents, ES256 over canonicalised JSON, each embedding the SHA-256 of its parent: **Intent** → **Cart** → **Payment**. Cart mandates are signed by both merchant and shopper. The chain is verified once when built and again immediately before money is requested, so a basket edited in between is caught.

A distinction that matters. “Under ₹5,000” is a statement about the *item*, not the total. A ₹4,299 shoe plus 18% GST is ₹5,072 — and refusing that would be obeying the letter of the instruction while breaking its meaning. The mandate carries both figures separately.

Payments

Razorpay in test mode behind a gateway interface, with a mock implementation that satisfies the same contract — so the whole purchase flow is runnable with no credentials. The Razorpay adapter refuses to start if the key isn’t a `rzp_test_` key. Refunds go back through the rails the charge came in on, and the stock decision differs by state: an order that never shipped returns its units, a delivered one does not, and a cancelled one already did.

The saved payment method table holds **no credentials** — no card number, CVV or token column exists, and a test asserts that. That is the whole safety argument: there is nothing to leak.

Agent protocols

The point of the platform is that other people’s agents can transact here, so the machine-readable surface is the product, not a side door.

PROTOCOL	SURFACE	WHAT IT DOES
MCP	<code>/api/mcp · studio</code>	Six tools over stateless JSON-RPC. No tool can charge — <code>prepare_purchase</code> returns an authorisation URL for a human.
UCP	<code>/.well-known/ucp</code>	Discovery plus a per-merchant manifest of capabilities, payment handlers and the merchant’s <i>public</i> signing key.
ACP	<code>/api/acp/.../feed.json</code>	The AI-readable product feed, JSON or CSV. This <i>is</i> the catalogue — nothing about it is hand-authored.
AP2	signed mandates	The Intent → Cart → Payment chain described above.
x402	<code>/api/x402/...</code>	HTTP 402 challenge and signed retry against a mock facilitator — no wallet, no funds.

The catalogue, and how it’s retrieved

503 products across 57 categories, generated rather than sampled — and the point is coherence, not volume. Each product is composed from an archetype (what it *is*) × a material (what it’s *made of*) × a brand tier (how well it’s made), and its description, its quality scores and its reviews are all written from those same three facts.

So a mesh trainer scores breathability 5 and water resistance 1, says exactly that in its description, and collects reviews complaining about wet feet. A dataset whose prose and attributes disagree is worse than no dataset: retrieval learns the false association and nothing downstream can see it.

BREATHABILITY SCORE	REVIEWS PRAISING AIRFLOW	REVIEWS COMPLAINING OF HEAT
1–2 (poor)	0	372

BREATHABILITY SCORE	REVIEWS PRAISING AIRFLOW	REVIEWS COMPLAINING OF HEAT
3 (unremarkable)	0	0
4–5 (good)	349	0

Opinion, prose and attributes agree because all three come from the same source.

Two indexes, deliberately

One vector per product answers *which product?* Reviews are embedded separately, one chunk each, answering *what do we know about it?* Folding forty opinions into a product’s own vector would average its spec sheet together with everyone’s experience of it and match neither well.

Rated features are a predicate, not a similarity

No embedding represents “at least 4 out of 5”, and a bi-encoder puts *waterproof but not breathable* almost exactly where it puts *waterproof and breathable*. So the model extracts the constraint from language — which is what it’s for — and SQL applies it as a pre-filter inside both retrieval legs, which is what that’s for.

Anti-hallucination floor. A query whose best semantic match falls below 0.34 returns nothing at all rather than the nearest unrelated product. The threshold is measured, never chosen, and re-measured whenever the catalogue changes materially — growing it from 184 to 503 products lifted the unstocked ceiling from 0.307 to 0.375, because a marketplace that now sells kitchen appliances genuinely *is* nearer to “washing machine”.

What was actually measured

Retrieval is scored against a labelled eval set whose ground truth is computed from the same quality scores that generated the catalogue — exact rather than hand-judged. Recall@10 and mean reciprocal rank, 78 cases:

QUERY KIND	CASES	RECALL@10	MRR	READS AS
Category	31	0.894	0.933	“formal shoes”
Single attribute	30	0.833	0.833	“waterproof shoes”
Trade-off	4	0.688	0.750	“waterproof but I know it won’t breathe”
Negation	1	1.000	1.000	“definitely not waterproof”
Budget	2	0.500	0.583	“comfortable shoes under 3000”
Paraphrase (<i>held out</i>)	10	0.280	0.300	“my feet get unbearably hot”
Overall	78	0.772	0.796	

The highlighted row is the honest one. Those ten cases describe a need *without naming the feature*, so no predicate can be extracted and retrieval has to work alone. They didn’t move when structured filtering was added — which is correct, and marks the ceiling of a 22-million-parameter embedder. That is where a larger model would earn its keep, and now there’s a number to prove whether it did.

The harness needed fixing before the system did. An early version capped expected results at 40 rows when 178 products qualified, so retrieval could return ten genuinely breathable shoes and score zero. Correcting the eval — changing no code — moved overall recall from 0.569 to 0.722. A wrong harness is worse than none: it sends you optimising against noise.

Stack and commands

Every dependency runs on a free tier without a credit card, and the whole application runs with *no API keys at all* through deterministic fallbacks — degraded in quality, never in function.

Application	Next.js 15 App Router, React 19, TypeScript — serving the UI and the agent protocol endpoints from one process
Database	Postgres 17 + pgvector via Drizzle ORM · 33 tables
Language model	Groq primary, failover to Gemini / OpenRouter / Cerebras. All inference remote
Embeddings	all-MiniLM-L6-v2, 384-dim, on-device — 22M parameters, ~4 ms a query
Payments	Razorpay test mode behind a gateway interface, with a mock implementation
Tests	Vitest, integration against the real database · 224 passing

COMMAND	DOES
<code>npm run doctor</code>	Preflight: database, pgvector, seed, index, embeddings, model, gateway
<code>npm run db:seed-catalog</code>	Generate the 503-product catalogue (idempotent)
<code>npm run db:seed-reviews</code>	Reviews and the purchase history the schema requires for them
<code>npm run catalog:index</code>	Rebuild product embeddings and review chunks
<code>npm run eval:retrieval</code>	Score retrieval against the labelled eval set
<code>npm run eval:relevance-gate</code>	Re-measure the anti-hallucination threshold
<code>npm run mcp:stdio</code>	Serve the MCP tools to a desktop agent client

Demo logins use `demo@shopper.test` and `care@stride.test`. Razorpay test card 4100 2800 0000 1007 — the international 4111... number declines.